Ch. 1 Notes

Arrays, Vectors, C-Strings

Vector declaration & init

```
vector<int> a;            // empty
vector<int> b(5);         // 5 zeros
vector<int> c(5, 7);      // 5 sevens
vector<int> d{1,2,3};     // literal list
vector<int> e = d;        // deep copy
```

Element access

`v[i]`	no bounds check (UB if bad)
`v.at(i)`	throws `out_of_range`
`v.front()`	same as `v[0]`
`v.back()`	same as `v[size-1]`
`v.size()`	returns `size_t` (unsigned!)

Operation complexity

`v[i]` / `at(i)`	$O(1)$	random access
`push_back`	$O(1)^*$	amortized
`pop_back`	$O(1)$	no shift
`back` / `front`	$O(1)$	
`insert(pos, x)`	$O(n)$	shift right
`erase(pos)`	$O(n)$	shift left
`resize(n)`	$O(n)$	may realloc
`reserve(n)`	$O(n)$	no default-init

Canonical loops

```
for (size_t i = 0; i < v.size(); ++i) //
     index
for (int x : v)
    // by value
for (int& x : v)
    // by ref (mutate)
for (const int& x : v)
    // read-only, no copy
```

Iteration patterns

Running best: track `best`, compare each element.
Running count: `++count` when predicate holds.
First match: early `return` / `break` on hit.
Paired scan (two vectors): single index drives both.
In-place transform: `v[i] = f(v[i])` – no new vector.

`resize` vs `push_back`

- `resize(n)` fills with default-constructed values; indexes `0..n-1` become valid immediately.
- `push_back(x)` appends one; size grows by 1.
- Know-the-size-up-front? Prefer `resize` then index. Size unknown? Use `push_back` (optionally `reserve`ing first to avoid reallocs).

Value semantics

- `b = a;` – **deep copy** of every element.
- `a == b` – element-wise compare; sizes must match.
- Pass large vectors by `const&` to avoid copies.
- After `push_back`/`resize`/`reserve`, *existing* iterators and pointers may be invalidated.

Swap

```
int t = a; a = b; b = t;    // by hand
std::swap(a, b);            // preferred
std::swap(v[i], v[j]);      // swap
    elements
```

Never `a = b; b = a;` – you lose a's old value.

Arrays vs. vectors

	array	**vector**
size	fixed at compile time	dynamic
passes size	no (decays to ptr)	yes (`.size()`)
bounds check	never	`.at()`
copy with =	no (just pointer)	yes (deep)
compare with ==	no (pointers)	yes (elements)
use when	fixed known N	default choice

2D containers

```
int m[3][4] = {};             // raw 2D
    array
vector<vector<int>> g(R, vector<int>(C,
    0));
```

Row-major: inner loop varies the *column*. Reversing nesting tanks cache performance.

C-strings vs. `std::string`

declare	`char s[N]` / `const char*`
terminator	`'\0'` required
compare	`strcmp(a,b)` returns `<0,0,>0`
concat	`strcat` (must pre-size)
assign	`strcpy(dst, src)` – *not* =
modern	`std::string` – all of the above with operators

`strcmp` gotcha: `if (strcmp(a,b))` tests "not equal."

<cctype> quick ref

`isalpha/isdigit/isalnum`	classify	All
`isspace`	space, tab, newline	
`isupper/islower`	case	
`toupper/tolower`	convert	

take `int`, return `int`. Cast `char` to `unsigned char` first to avoid UB on negative values.

Top gotchas

- `v.size() - 1` when `v` is empty: underflow to huge unsigned.
- Modifying a vector while iterating invalidates iterators.
- `strcmp` returns 0 on *equality* (opposite of what "truthy" suggests).
- Row-major 2D: iterate rows outer, cols inner.
- Parallel vectors drift out of sync; use a struct.
- `vector` grows but never auto-shrinks. `shrink_to_fit()` is non-binding; force release via `vector<T>().swap(v)`.